



Saint Columban College
College of Computing Studies
Pagadian City



Flag Hunters

Flag Hunters 



Easy

Reverse Engineering

picoCTF 2025

browser_webshell_solvable

AUTHOR: SYREAL

Description

Lyrics jump from verses to the refrain kind of like a subroutine call. There's a hidden refrain this program doesn't print by default. Can you get it to print it? There might be something in it for you.

The program's source code can be downloaded [here](#).

Additional details will be available after launching your challenge instance.

This challenge launches an instance on demand.

Its current status is: **NOT_RUNNING**

Launch Instance

Hints 

1 2 3

23,183 users solved



88% Liked



 picoCTF{FLAG}

Submit Flag

Author: The Analyst: Hyposelenia

Challenge: Flag Hunters

Category: Reverse Engineering

Date: 04/03/26



I. Objective

The objective of this challenge is to analyze a Python program and understand how **user input can affect program behavior**.

The goal is to identify a weakness in how input is handled and use it to **manipulate execution flow and reveal the flag**.

II. Background

Programs often accept input from users. If this input is not properly controlled or validated, it can be abused to:

- change how the program runs
- skip intended logic
- reveal hidden data

This type of issue is related to **input injection**, where a user inserts special input to alter program behavior.

III. Tool Used

- **Kali Linux (Oracle VirtualBox)** - Working environment
- **Netcat (nc)** - Used to connect to the remote service
- **Sublime Text (subl)** - Used to read and analyze the Python script

IV. Methodology

1. Launched **Kali Linux**
2. Downloaded the Python file using **wget**

```
(kali@kali) - [~/Downloads]
$ wget https://challenge-files.picoctf.net/c_verbal_sleep/f9515a5a3a8eda47
--2026-04-03 04:44:50-- https://challenge-files.picoctf.net/c_verbal_sleep/
Resolving challenge-files.picoctf.net (challenge-files.picoctf.net)... 3.165
Connecting to challenge-files.picoctf.net (challenge-files.picoctf.net)|3.165
HTTP request sent, awaiting response... 200 OK
Length: 3381 (3.3K) [application/octet-stream]
Saving to: 'lyric-reader.py'

lyric-reader.py                                     100%[=====]
2026-04-03 04:44:51 (153 MB/s) - 'lyric-reader.py' saved [3381/3381]
```



3. Opened and analyzed the script using: **subl <filename>.py**

```
(kali㉿kali)-[~/Downloads]
$ subl lyric-reader.py
(kali㉿kali)-[~/Downloads]
$
```

4. Observed how the program processes song lyrics and user input

```
1 import re
2 import time
3
4
5 # Read in flag from file
6 flag = open('flag.txt', 'r').read()
7
8 secret_intro = \
9 '''Pico warriors rising, puzzles laid bare,
10 Solving each challenge with precision and flair.
11 With unity and skill, flags we deliver,
12 The ether's ours to conquer, '''\
13 + flag + '\n'
14
15
16 song_flag_hunters = secret_intro +\
17 '''
18
19 [REFRAIN]
20 We're flag hunters in the ether, lighting up the grid,
21 No puzzle too dark, no challenge too hid.
22 With every exploit we trigger, every byte we decrypt,
23 We're chasing that victory, and we'll never quit.
24 CROWD (Singalong here!);
25 RETURN
26
27 [VERSE1]
28 Command line wizards, we're starting it right,
29 Spawning shells in the terminal, hacking all night.
30
```

5. Noticed this critical part:

```
elif re.match(r"CROWD.*", line):
    crowd = input('Crowd: ')
    song_lines[lip] = 'Crowd: ' + crowd
    lip += 1
elif re.match(r"RETURN [0-9]+", line):
    lip = int(line.split()[1])
```

6. Launched the challenge instance

Instance Time Remaining: 14:50

Restart Instance



7. Connected using: **nc <host> <port>**

```
$ nc verbal-sleep.picoctf.net 56943
```

8. When prompted for input (Crowd:), entered: **;RETURN 0**

```
NO puzzle too dark, NO chance  
With every exploit we trigger  
We're chasing that victory, a  
Crowd: ;RETURN 0  
  
Echoes in memory, packets in  
Digging through the remnants
```

9. The program's execution flow was manipulated
10. The flag was revealed

V. Result

picoCTF{70637h3r_f0r3v3r_0ed60683}

```
Crowd:  
Pico warriors rising, puzzles laid bare,  
Solving each challenge with precision and flair.  
With unity and skill, flags we deliver,  
The ether's ours to conquer, picoCTF{70637h3r_f0r3v3r_0ed60683}
```

VI. Explanation

What is happening in this program?

The program is like a **song player**

- It reads lines
- Prints them one by one



- Sometimes asks the user to join (Crowd input)

Think of it like a game:

- The program is following instructions
- Like: “go here → print this → go there”

The Weakness:

Look at this:

song_lines[lip] = 'Crowd: ' + crowd

It takes **your input** and puts it directly into the program

Why is this dangerous?

Because the program later does this:

for line in song_lines[lip].split(';'):

It splits lines using:

Semicolon (;)

So, what does that mean then?

Hello

It becomes:

Crowd: Hello

But if you type:

;RETURN 0

It becomes:

Crowd: ;RETURN 0

Then .split(';') turns it into:

["Crowd: ", "RETURN 0"]

BOOM! Here's the trick

Now the program sees:

RETURN 0

And this part runs:



```
elif re.match(r"RETURN [0-9]+", line):  
lip = int(line.split()[1])
```

It means:

“Jump to line 0”

What is line 0?

Line 0 contains:

```
secret_intro = "... " + flag + "\n"
```

This includes the flag

What is this vulnerability?

This is similar to:

Command Injection

VII. Conclusion

This challenge demonstrates how improper handling of user input can lead to serious vulnerabilities.

By injecting crafted input, the program’s execution flow was altered, allowing access to hidden data.

This reinforces a key concept in cybersecurity:

Never trust user input — always validate and sanitize it.

— **The Analyst: Hyposelenia**